

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 1- CASE STUDY ANALYSIS

Course Code: DSA0504

**Course Title: Query Processing for
Data Science**

CO Assessed: CO4

**Assessment: Tool 1 – Case Study
Analysis**

Weightage: 40

Total Marks: 20 Marks

CO3 Statement: Explain the fundamentals of data wrangling, its importance, processes, and tools. Understand different data formats such as CSV, JSON, and XML along with machine-readable data. (BL4, BL5 and BL6)

Student Name: Jeya Harshini R

Reg. No.:192424051

Section / Batch:2024

Date of Submission:10/8/26

Faculty In-charge: Dr. R.Mahesh Muthulakshmi

Project Mode: Individual Submission

1. INTRODUCTION

Data wrangling is the process of collecting, inspecting, cleaning, transforming, and preparing raw data for reliable analysis. In a retail environment, sales data may be collected from multiple stores and systems, resulting in inconsistent product names, duplicate transactions, inconsistent categories, and missing product information.

This case study focuses on a retail chain where data-quality issues affect the accuracy of sales reports and inventory planning. A systematic data-wrangling process is therefore required to clean the data and produce reliable business information.

2. CASE STUDY BACKGROUND

The retail chain collects sales information from multiple stores. However, differences in the way products and transactions are recorded create inconsistencies in the centralized sales data.

The major issues identified in the case are:

- Product names are entered differently.
- Duplicate transactions are created because of repeated uploads.
- Product categories are inconsistent.
- Some product information is missing.

For example, the same product may be recorded as:

Laptop

Laptop PC

Notebook Computer

These differences make it difficult to group sales correctly and can result in inaccurate sales reports and inventory decisions.

3. TASK 1 – IDENTIFICATION AND ANALYSIS OF DATA QUALITY ISSUES

3.1 Inconsistent Product Names

The same product may be represented using different names, such as:

Laptop

Laptop PC

Notebook Computer

Although these names may refer to the same or related product, a computer-based system may treat them as separate values.

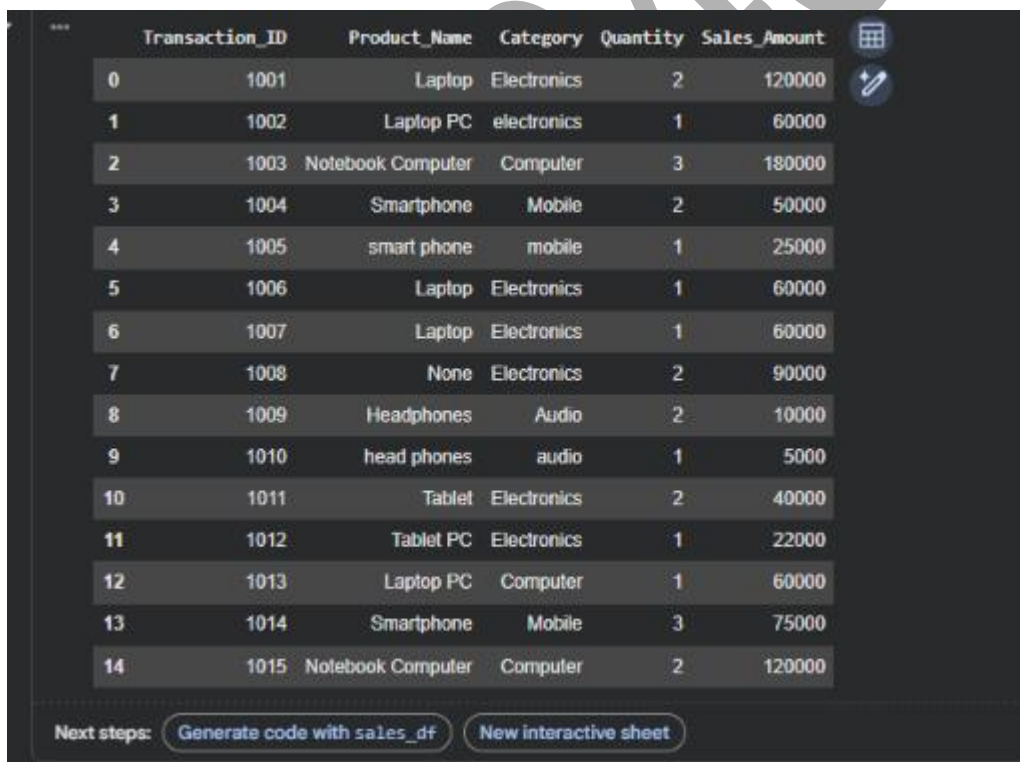
Impact

This can result in:

- Incorrect product counts
- Splitting of sales across multiple product names
- Inaccurate sales summaries
- Difficulty in inventory analysis

Practical Demonstration

The sample retail dataset created in Python contains multiple product-name representations.



	Transaction_ID	Product_Name	Category	Quantity	Sales_Amount
0	1001	Laptop	Electronics	2	120000
1	1002	Laptop PC	electronics	1	60000
2	1003	Notebook Computer	Computer	3	180000
3	1004	Smartphone	Mobile	2	50000
4	1005	smart phone	mobile	1	25000
5	1006	Laptop	Electronics	1	60000
6	1007	Laptop	Electronics	1	60000
7	1008	None	Electronics	2	90000
8	1009	Headphones	Audio	2	10000
9	1010	head phones	audio	1	5000
10	1011	Tablet	Electronics	2	40000
11	1012	Tablet PC	Electronics	1	22000
12	1013	Laptop PC	Computer	1	60000
13	1014	Smartphone	Mobile	3	75000
14	1015	Notebook Computer	Computer	2	120000

Next steps: [Generate code with sales_df](#) [New interactive sheet](#)

3.2 Duplicate Transactions

Duplicate transactions can occur when the same sales data is uploaded more than once.

For example, two records may contain the same:

- Product
- Category
- Quantity
- Sales amount

even though they have different transaction identifiers.

Impact

Duplicate transactions can:

- Overstate total sales
- Increase reported quantities
- Distort product performance
- Affect inventory planning

Therefore, duplicate records should be detected and investigated before removal.

```
[?] 0s
duplicate_transactions = sales_df[
    sales_df.duplicated(
        subset=[
            "Product_Name",
            "Category",
            "Quantity",
            "Sales_Amount"
        ],
        keep=False
    )
]

print("Potential duplicate transactions:")
display(duplicate_transactions)
```

Potential duplicate transactions:

Transaction_ID	Product_Name	Category	Quantity	Sales_Amount	
5	1006	Laptop	Electronics	1	60000
6	1007	Laptop	Electronics	1	60000

Next steps: [Generate code with duplicate_transactions](#) [New interactive sheet](#)

3.3 Inconsistent Product Categories

The same category may be entered using different capitalization or naming conventions.

For example:

Electronics

electronics

Computer

Mobile

mobile

Audio

audio

Such differences can cause the same category to be treated as separate categories during analysis.

Impact

This can result in:

- Incorrect category-level sales totals
- Fragmented reports
- Difficulty comparing product categories
- Inaccurate business analysis

```
[5] print("Unique Categories:")
    print(
        sales_df["Category"]
        .dropna()
        .unique()
    )

--- Unique Categories:
    ['Electronics' 'electronics' 'Computer' 'Mobile' 'mobile' 'Audio' 'audio']

[6] sales_df["Category_Standardized"] = (
    sales_df["Category"]
    .astype("string")
    .str.strip()
    .str.title()
)

print("\nStandardized Categories:")
print(
    sales_df["Category_Standardized"]
    .dropna()
    .unique()
)

--- Standardized Categories:
    <StringArray>
    ['Electronics', 'Computer', 'Mobile', 'Audio']
    Length: 4, dtype: string
```

3.4 Missing Product Information

Some transactions may not contain a product name.

Missing product information reduces the completeness of the sales dataset and makes it difficult to determine which product generated a particular transaction.

Impact

Missing product information can affect:

- Product-level sales analysis
- Inventory planning
- Product performance comparison
- Business reporting

The missing value should be identified and investigated rather than replaced with an invented product name.

```
[4] ✓ On missing_values = sales_df.isnull().sum()

print("Missing Values:")
print(missing_values)

Missing Values:
Transaction_ID    0
Product_Name      1
Category          0
Quantity          0
Sales_Amount      0
dtype: int64

[5] ✓ On missing_records = sales_df[
    sales_df["Product_Name"].isnull()
]

print("\nRecords with missing Product_Name:")
display(missing_records)

Records with missing Product_Name:
  Transaction_ID  Product_Name  Category  Quantity  Sales_Amount
7             1008         None  Electronics         2         90000

[6] ✓ On missing_records = sales_df[
    sales_df["Product_Name"].isnull()
]

print("\nRecords with missing Product_Name:")
display(missing_records)

Records with missing Product_Name:
  Transaction_ID  Product_Name  Category  Quantity  Sales_Amount
7             1008         None  Electronics         2         90000
```

4. TASK 2 – FUZZY MATCHING OF PRODUCT NAMES

Fuzzy matching is a technique used to identify strings that are similar even when they are not exactly identical.

It is useful in this case because product names may differ due to abbreviations, spacing, or alternative naming.

Examples include:

Laptop → Laptop PC

Smartphone → smart phone

Headphones → head phones

A fuzzy-matching algorithm can calculate a similarity score between product names and identify potential matches for further investigation.

4.1 Practical Fuzzy Matching

The Python RapidFuzz library was used to demonstrate similarity matching between product names.

The practical output showed potential matches between similar product names.

```
[11] ✓ 0s from rapidfuzz import process, fuzz
      print("RapidFuzz installed successfully.")
      RapidFuzz installed successfully.

[12] ✓ 0s product_names = (
      sales_df["Product_Name"]
      .dropna()
      .astype(str)
      .str.strip()
      .unique()
      )

      print("Unique product names:")
      print(product_names)
      Unique product names:
      ['Laptop' 'Laptop PC' 'Notebook Computer' 'Smartphone' 'smart phone'
      'Headphones' 'head phones' 'Tablet' 'Tablet PC']

[13] ✓ 0s product_names = (
      sales_df["Product_Name"]
      .dropna()
      .astype(str)
      .str.strip()
      .unique()
      )

      print("Unique product names:")
      print(product_names)
      Unique product names:
      ['Laptop' 'Laptop PC' 'Notebook Computer' 'Smartphone' 'smart phone'
      'Headphones' 'head phones' 'Tablet' 'Tablet PC']
```

Analysis

The fuzzy-matching results demonstrate that exact string matching is not always sufficient for retail data. Similarity-based matching can identify candidate matches even when product names contain differences in spelling, spacing, or wording.

However, fuzzy matching should be treated as a **candidate-identification technique** rather than automatically assuming that every similar name represents the same product. Final standardization should be based on business rules or an approved product master.

5. TASK 3 – DATA CLEANUP PROCESS

A systematic data-cleaning workflow was designed for the retail sales data.

5.1 Data Collection

Sales data is collected from the different retail stores or source systems.

5.2 Data Inspection

The structure, columns, data types, record counts, and sample records are inspected before cleaning.

Python and Pandas were used to inspect the sample dataset.

```
[3] ✓ Os
print("Dataset Shape:", sales_df.shape)

print("\nColumn Names:")
print(sales_df.columns.tolist())

print("\nData Types:")
print(sales_df.dtypes)

print("\nFirst 5 Records:")
display(sales_df.head())
```

Dataset Shape: (15, 5)

Column Names:
['Transaction_ID', 'Product_Name', 'Category', 'Quantity', 'Sales_Amount']

Data Types:
Transaction_ID int64
Product_Name object
Category object
Quantity int64
Sales_Amount int64
dtype: object

First 5 Records:

	Transaction_ID	Product_Name	Category	Quantity	Sales_Amount
0	1001	Laptop	Electronics	2	120000
1	1002	Laptop PC	electronics	1	60000
2	1003	Notebook Computer	Computer	3	180000
3	1004	Smartphone	Mobile	2	50000
4	1005	smart phone	mobile	1	25000

5.3 Missing-Value Detection

Missing product information is identified using Pandas.

The missing records are retained for investigation rather than being immediately deleted.

5.4 Duplicate Detection

Potential duplicate transactions are identified using transaction attributes such as:

- Product name
- Category
- Quantity
- Sales amount

Only confirmed duplicate records should be removed.

```
[18] 0s before = len(sales_df)

sales_cleaned = sales_df.drop_duplicates(
    subset=[
        "Product_Name_Standardized",
        "Category_Standardized",
        "Quantity",
        "Sales_Amount"
    ],
    keep="first"
).copy()

after = len(sales_cleaned)

print("Records before duplicate removal:", before)
print("Records after duplicate removal:", after)
print("Duplicate records removed:", before - after)
```

```
--- Records before duplicate removal: 15
Records after duplicate removal: 13
Duplicate records removed: 2
```

5.5 Product Name Standardization

Potentially equivalent product names are investigated using fuzzy matching.

A controlled mapping is then applied to standardize confirmed product names.

For example:

Original Name	Standardized Name
Laptop	Laptop

Original Name	Standardized Name
---------------	-------------------

Laptop PC	Laptop
-----------	--------

Notebook Computer Laptop	
--------------------------	--

Smartphone	Smartphone
------------	------------

smart phone	Smartphone
-------------	------------

Headphones	Headphones
------------	------------

head phones	Headphones
-------------	------------

Tablet PC	Tablet
-----------	--------

5.6 Category Standardization

Category names are standardized by removing unnecessary spaces and applying a consistent capitalization convention.

For example:

electronics

Electronics

can be represented consistently as:

Electronics

```
[sc] 0s
sales_df["Category_Standardized"] = (
    sales_df["Category"]
    .astype("string")
    .str.strip()
    .str.title()
)

display(
    sales_df[
        [
            "Category",
            "Category_Standardized"
        ]
    ]
)
```

	Category	Category_Standardized
0	Electronics	Electronics
1	electronics	Electronics
2	Computer	Computer
3	Mobile	Mobile
4	mobile	Mobile
5	Electronics	Electronics
6	Electronics	Electronics
7	Electronics	Electronics
8	Audio	Audio
9	audio	Audio
10	Electronics	Electronics
11	Electronics	Electronics
12	Computer	Computer
13	Mobile	Mobile
14	Computer	Computer

5.7 Handling Missing Product Information

Missing product names were represented as:

Unknown Product

rather than inventing a product name.

This allows the record to remain in the dataset while clearly indicating that the original product information is unavailable.

5.8 Final Clean Dataset

After the cleaning operations, a standardized dataset was produced containing consistent product names and categories.

```
[28] ✓ 0s final_columns = [
    "Transaction_ID",
    "Product_Name_Standardized",
    "Category_Standardized",
    "Quantity",
    "Sales_Amount"
]

cleaned_df = sales_cleaned[final_columns].copy()

cleaned_df = cleaned_df.rename(columns={
    "Product_Name_Standardized": "Product_Name",
    "Category_Standardized": "Category"
})

display(cleaned_df)
```

	Transaction_ID	Product_Name	Category	Quantity	Sales_Amount
0	1001	Laptop	Electronics	2	120000
1	1002	Laptop	Electronics	1	60000
2	1003	Laptop	Computer	3	180000
3	1004	Smartphone	Mobile	2	50000
4	1005	Smartphone	Mobile	1	25000
7	1008	Unknown Product	Electronics	2	90000
8	1009	Headphones	Audio	2	10000
9	1010	Headphones	Audio	1	5000
10	1011	Tablet	Electronics	2	40000
11	1012	Tablet	Electronics	1	22000
12	1013	Laptop	Computer	1	60000
13	1014	Smartphone	Mobile	3	75000
14	1015	Laptop	Computer	2	120000

Next steps: [Generate code with cleaned_df](#) [New interactive sheet](#)

6. BEFORE AND AFTER COMPARISON

The dataset was compared before and after cleaning to evaluate the effect of the data-wrangling process.

```
[21] comparison = pd.DataFrame({
  "Measure": [
    "Total Records",
    "Unique Product Names",
    "Missing Product Names"
  ],

  "Before Cleaning": [
    len(sales_df),
    sales_df["Product_Name"].nunique(),
    sales_df["Product_Name"].isna().sum()
  ],

  "After Cleaning": [
    len(cleaned_df),
    cleaned_df["Product_Name"].nunique(),
    cleaned_df["Product_Name"].isna().sum()
  ]
})

display(comparison)
```

	Measure	Before Cleaning	After Cleaning
0	Total Records	15	13
1	Unique Product Names	9	5
2	Missing Product Names	1	0

Next steps: [Generate code with comparison](#) [New interactive sheet](#)

The comparison demonstrates that data cleaning reduces inconsistencies in product representation and makes the dataset more suitable for reliable aggregation and reporting.

The objective is not simply to reduce the number of records. Instead, the objective is to ensure that the remaining records accurately represent the underlying retail transactions.

7. TASK 4 – PYTHON AUTOMATION FOR DATA CLEANING

Python can automate repeated data-cleaning operations and reduce manual effort.

The workflow implemented in the practical demonstration was:



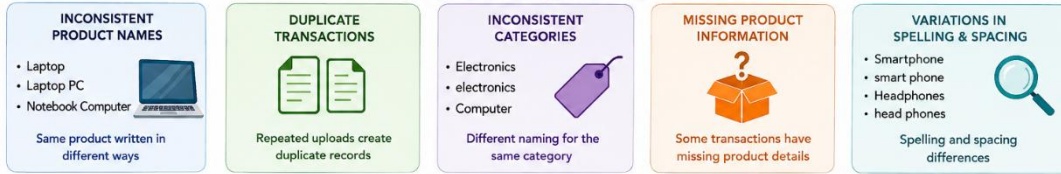
RETAIL SALES REPORTING SYSTEM

DATA WRANGLING FOR ACCURATE SALES REPORTING



Poor quality data leads to inaccurate sales reports, wrong inventory planning, and poor business decisions.

COMMON DATA QUALITY ISSUES



DATA WRANGLING PROCESS



BEFORE CLEANING (RAW DATA)				
Transaction_ID	Product_Name	Category	Quantity	Sales_Amount
1001	Laptop	Electronics	2	120000
1002	Laptop PC	electronics	1	60000
1003	Notebook Computer	Computer	3	180000
1004	Smartphone	Mobile	2	50000
1005	smart phone	mobile	1	25000
1006	Laptop	Electronics	1	60000
1007	Laptop	Electronics	1	60000
1008	NULL	Electronics	2	90000
1009	Headphones	Audio	2	10000
1010	head phones	audio	1	5000
...	...	...	...	...

➔ Data Wrangling & Cleaning

AFTER CLEANING (CLEAN DATA)				
Transaction_ID	Product_Name	Category	Quantity	Sales_Amount
1001	Laptop	Electronics	3	180000
1003	Laptop	Electronics	3	180000
1004	Smartphone	Mobile	3	75000
1009	Headphones	Audio	3	15000
1011	Tablet	Electronics	2	40000
1012	Tablet	Electronics	1	22000
1014	Unknown Product	Electronics	2	90000
...	...	...	...	...

BUSINESS BENEFITS

- Accurate sales reporting
- Better inventory planning
- Improved product performance analysis
- Reduced reporting errors & manual effort
- Better business decisions & customer satisfaction

Clean, consistent, and reliable data leads to accurate insights and improved retail performance.

Python tools used

Tool / Technique Purpose

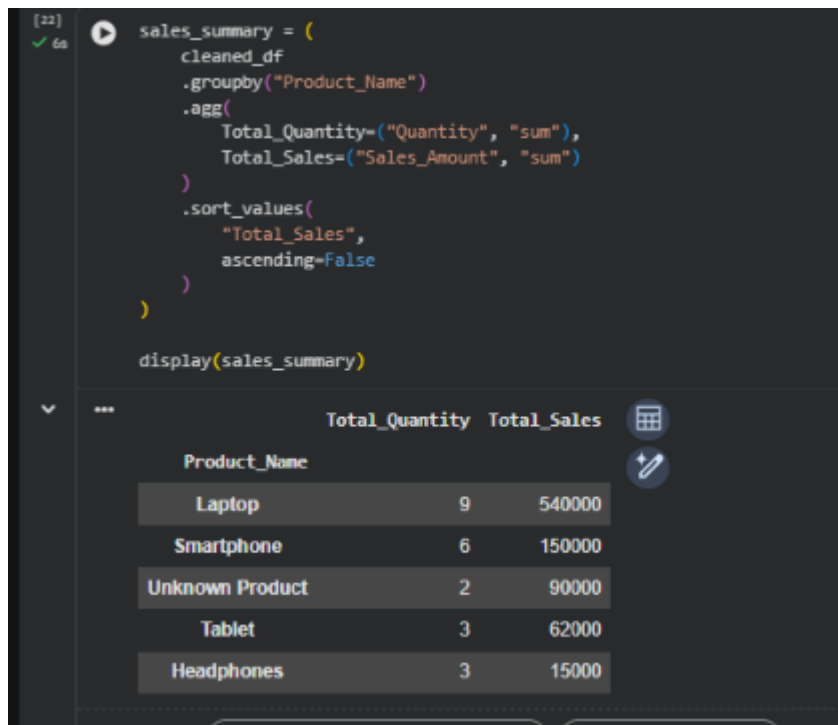
Python	Overall automation
Pandas	Data loading and cleaning
NumPy	Data processing support
RapidFuzz	Fuzzy string matching
isnull()	Missing-value detection
uplicated()	Duplicate detection
String operations	Standardization
groupby()	Sales aggregation
Matplotlib	Visualization

Python is suitable because the cleaning process can be repeated consistently whenever new sales data is received.

8. SALES ANALYSIS AFTER CLEANING

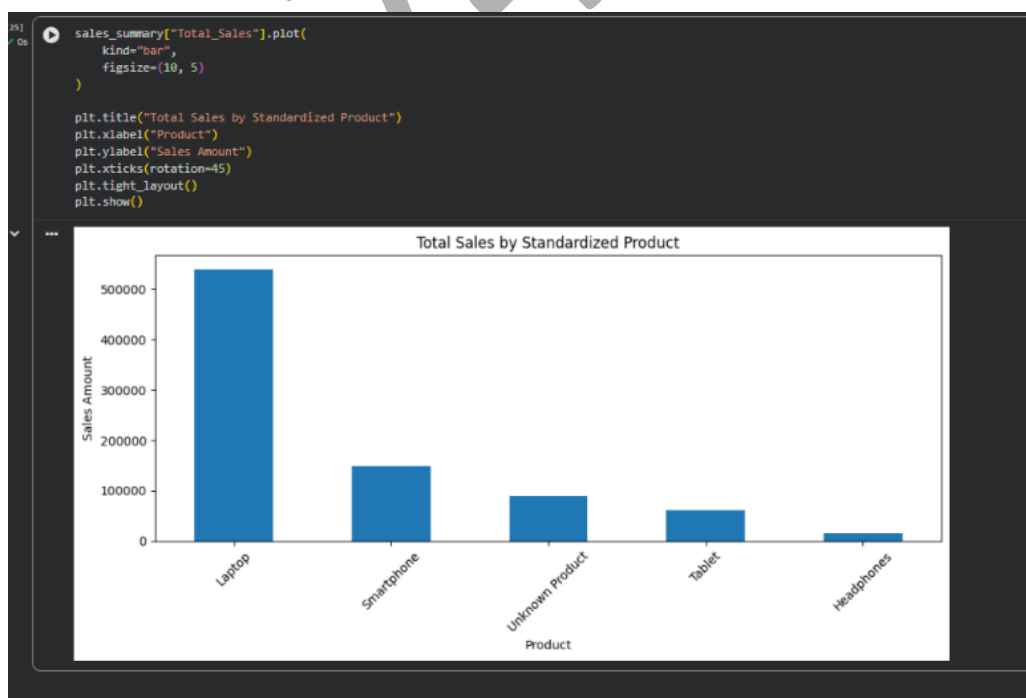
After standardizing product names, sales were grouped by the standardized product name.

The total quantity and total sales amount were calculated for each product.



9. SALES VISUALIZATION

A bar chart was generated to visualize total sales for the standardized products.



Analysis

The visualization provides a clearer comparison of product-level sales after inconsistent product names have been standardized.

Without standardization, sales belonging to the same product could be split across different names, potentially producing misleading product-performance results.

10. TASK 5 – BUSINESS BENEFITS OF CLEAN DATA

Clean and integrated retail data provides several benefits.

10.1 Accurate Sales Reporting

Standardized product names prevent sales from being incorrectly divided across multiple names.

10.2 Improved Inventory Planning

Reliable product-level sales information helps businesses understand which products have higher demand.

10.3 Better Product Performance Analysis

Standardized categories and product names allow products to be compared more accurately.

10.4 Reduced Reporting Errors

Removing confirmed duplicate transactions prevents sales totals from being artificially inflated.

10.5 Faster Data Processing

Python automation reduces repetitive manual cleaning work when new sales data is received.

10.6 Better Business Decisions

Reliable sales information supports more informed decisions related to inventory, product demand, and store operations.

The faculty case specifically identifies inaccurate sales reports and inventory planning as consequences of the data-quality problems.

11. OVERALL ANALYSIS

The main challenge in the retail sales case is not simply the presence of incorrect values. The larger issue is that the same business information can be represented differently across transactions.

For example, if Laptop, Laptop PC, and Notebook Computer are treated as completely separate products, the sales associated with those records may be divided between multiple groups. Similarly, duplicate transactions can artificially increase sales totals.

The practical demonstration shows that a combination of **duplicate detection, missing-value analysis, fuzzy matching, standardization, and aggregation** can improve the consistency of retail data.

Fuzzy matching is particularly useful for identifying potential product-name similarities. However, similarity scores should support human or business-rule validation rather than automatically changing every similar value.

The cleaned dataset can then be used to generate more reliable sales summaries and visualizations.

12. CONCLUSION

The Retail Sales Reporting System case demonstrates how data-quality problems can affect the reliability of business reports.

The major issues identified were **inconsistent product names, duplicate transactions, inconsistent product categories, and missing product information**. A systematic data-wrangling workflow was developed to identify and address these issues.

Python, Pandas, RapidFuzz, and Matplotlib were used in the practical demonstration to inspect, clean, standardize, analyze, and visualize the retail sales data.

Fuzzy matching helped identify potentially similar product names, while controlled standardization ensured that confirmed equivalent products could be grouped consistently.

The resulting cleaned data provides a stronger foundation for sales reporting, product analysis, inventory planning, and business decision-making.

13. FINAL WORKFLOW

